Aprendizaje. Árboles de Decisión

(Basado parcialmente en el libro de R & N)

E. López Rubio, E. Domínguez y J. L. Pérez de la Cruz

Univ. de Málaga

Año 2025/26

Contenidos

1. Algunos conceptos básicos en Aprendizaje Automático
2. Árboles de Decisión y algoritmo ID3
3. Rendimiento, Entrenamiento y Aprendizaje
4. Conclusión

Aspectos destacados

- Definimos *aprendizaje*, *clasificación* y *regresión*.
- Presentamos un método para resolver problemas de clasificación: *Árboles de decisión*.
- Enfatizamos la diferencia *entrenamiento vs. aprendizaje*

Algunos conceptos básicos en Aprendizaje Automático

Motivación

Algunas palabras presentes en el habla cotidiana:

- Big Data
- Deep learning
- ...

Todas ellas están relacionadas con el *Aprendizaje Automático* o *Aprendizaje de Máquinas*.

El hardware potente y la recolección masiva de datos han hecho posible el Aprendizaje Automático a gran escala y están en la raíz de los grandes éxitos recientes de la IA.

Aprendizaje

Decimos que un agente aprende si

- Realiza observaciones sobre el mundo,
- procesa estas observaciones, y
- mejora su rendimiento en una tarea.

Por ejemplo, considere el famoso sistema que aprende a jugar Breakout:

`https://www.youtube.com/watch?v=V1eYniJ0Rnk`

- El agente *observa* el estado de la pantalla y la acción seleccionada;
- La *Tarea* es destruir el muro;
- El *Rendimiento* se mide por el número de puntos alcanzados.

En este ejemplo, el rendimiento mejora: el agente "aprende".

Aprendizaje Supervisado

Cuando el agente observa algunos ejemplos de pares entrada-salida y aprende una función que mapea de entrada a salida llamamos a esto

APRENDIZAJE SUPERVISADO

En este curso consideraremos solo aprendizaje supervisado

Aprendizaje No Supervisado

Cuando no se suministra una salida explícita (es decir, la respuesta correcta para cada caso no se proporciona) y aun así el agente detecta patrones en la entrada llamamos a esto

APRENDIZAJE NO SUPERVISADO

Ejemplos son la detección de reglas de asociación entre atributos, o el agrupamiento de casos en clases similares.

Cuando la respuesta "correcta" para cada caso no se proporciona, pero se da una recompensa numérica tras algunas acciones del agente, llamamos a esto

APRENDIZAJE POR REFUERZO

El vídeo recién visto es un ejemplo de esto.

Aprendizaje Supervisado: ejemplo de un conjunto de datos

Example	Sky	Temperature	Humidity	Wind	PlayTennis
$\mathbf{x}_1$	Sunny	High	High	Weak	-
$\mathbf{x}_2$	Sunny	High	High	Strong	-
$\mathbf{x}_3$	Cloudy	High	High	Weak	+
$\mathbf{x}_4$	Rainy	Warm	High	Weak	+
$\mathbf{x}_5$	Rainy	Low	Normal	Weak	+
$\mathbf{x}_6$	Rainy	Low	Normal	Strong	-
$\mathbf{x}_7$	Cloudy	Low	Normal	Strong	+
$\mathbf{x}_8$	Sunny	Warm	High	Weak	-
$\mathbf{x}_9$	Sunny	Low	Normal	Weak	+
$\mathbf{x}_{10}$	Rainy	Warm	Normal	Weak	+
$\mathbf{x}_{11}$	Sunny	Warm	Normal	Strong	+
$\mathbf{x}_{12}$	Cloudy	Warm	High	Strong	+
$\mathbf{x}_{13}$	Cloudy	High	Normal	Weak	+
$\mathbf{x}_{14}$	Rainy	Warm	High	Strong	-

Aprendizaje Supervisado: ejemplo de un conjunto de datos (cont.)

La diapositiva 9 muestra un ejemplo típico de un *conjunto de datos*, es decir, un conjunto de ejemplos proporcionados a un algoritmo de aprendizaje.

Cada fila es un *ítem* del conjunto de datos. En este conjunto de datos hay 14 ítems.

Cada columna corresponde a una variable o *atributo*. Los atributos en el ejemplo son *Cielo*, *Temperatura*, *Humedad*, *Viento*, y *JugarTenis*.

Para cada ítem, cada atributo tiene un *valor*. Este valor debe pertenecer a un dominio dado: *Cielo* puede ser soleado/nublado/lluvioso. *Temperatura* puede ser alta/templada/baja. *Humedad* puede ser alta/normal. *Viento* puede ser débil/fuerte. *JugarTenis* puede ser sí/no.

Uno de los atributos es el atributo de *salida*, *objetivo* o *clase*: en este conjunto de datos es *JugarTenis*. Los demás son atributos de *entrada*.

Queremos encontrar una regla (una función) que para cada 4-tupla de valores (cielo, temperatura, humedad, viento) de atributos de entrada prediga el valor del atributo de salida *JugarTenis*.

Aprendizaje Supervisado: definición formal

Un *conjunto de datos* es un conjunto $E = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)\}$ compuesto de N pares.
Cada par $(\boldsymbol{x}_i, y_i)$ es un ejemplo, punto, caso, o *ítem*, y consiste de una *entrada* $\boldsymbol{x}_i = (x_{i,1}, \ldots, x_{i,n})$ y una *salida* y_i.
La entrada es un vector n-dimensional y la salida es un valor único. A veces escribimos el ítem como un único vector $(x_{i,1}, \ldots, x_{i,n}, y_i)$ de $n+1$ dimensiones.
El *espacio de entrada* $\mathcal{E}$ es el conjunto de todas las entradas posibles.
Los componentes de la entrada pueden ser variables booleanas, categóricas o reales.
La salida también puede ser booleana, categórica o real. El *espacio de salida* $\mathcal{Y}$ es el conjunto de todas las salidas posibles.

Aprendizaje Supervisado: definición formal (cont.)

Se supone que

- existe una función desconocida $f : \mathcal{E} \to \mathcal{Y}$ que aplicada a cualquier $\boldsymbol{x} \in \mathcal{E}$ produce una salida $y = f(\boldsymbol{x})$;
- el conjunto de datos E es una muestra de $\{(\boldsymbol{x}, f(\boldsymbol{x})) \mid \boldsymbol{x} \in \mathcal{E}\}$
- se proporciona un cierto conjunto $\mathcal{H}$ de funciones h. Se llama *espacio de hipótesis* y sus miembros h se llaman *hipótesis*.

El problema del aprendizaje supervisado es: **Dado un conjunto de datos E, encontrar una hipótesis $h \in \mathcal{H}$ tal que h aproxime f en $\mathcal{E}$.**

En IA y Aprendizaje Automático, el proceso de encontrar (iterativamente) tal h se llama *aprendizaje*.

¿Cómo se hace? La idea es

- Aproximar f mediante una hipótesis h en el conjunto de datos E, donde los valores $y = f(\boldsymbol{x})$ son conocidos.
- Luego verificar de alguna manera si h *generaliza* bien, es decir, si h aproxima bien f fuera de E.

Aprendizaje Supervisado: tipos

Teniendo en cuenta la naturaleza del espacio de salida $\mathcal{Y}$, se definen varios tipos de aprendizaje supervisado:

- Si $\mathcal{Y}$ es (un intervalo de) el conjunto de números reales $\mathbb{R}$, se llama *regresión*.
- Si $\mathcal{Y}$ es categórico, es decir, un conjunto finito de etiquetas $\{v_1, \ldots, v_M\}$, se llama *clasificación*.
  - Si el tamaño del conjunto de etiquetas es 2, es una *clasificación binaria*
  - De lo contrario, es una *clasificación multiclase*.

¿Por qué decimos "clasificación"? Porque calcular $h(\boldsymbol{x})$ puede verse como asignar una etiqueta a $\boldsymbol{x}$, y una etiqueta puede verse como el identificador de una *clase*. Más formalmente, cada etiqueta v_i define un conjunto $C_i = \{x \mid f(\boldsymbol{x}) = v_i\}$ y la familia de todos los conjuntos $C_1, \ldots, C_m$ es una partición del espacio de entrada. Entonces cada C_i puede llamarse una clase.

Árboles de Decisión y algoritmo ID3

Árboles de decisión

Un *árbol de decisión* (AD) es la representación gráfica de una hipótesis de un cierto tipo.

En general, en los AD, tanto la entrada como la salida pueden ser booleanas, categóricas o continuas. Pero en este curso no consideraremos valores continuos.

La función representada por el árbol se define de la siguiente manera:

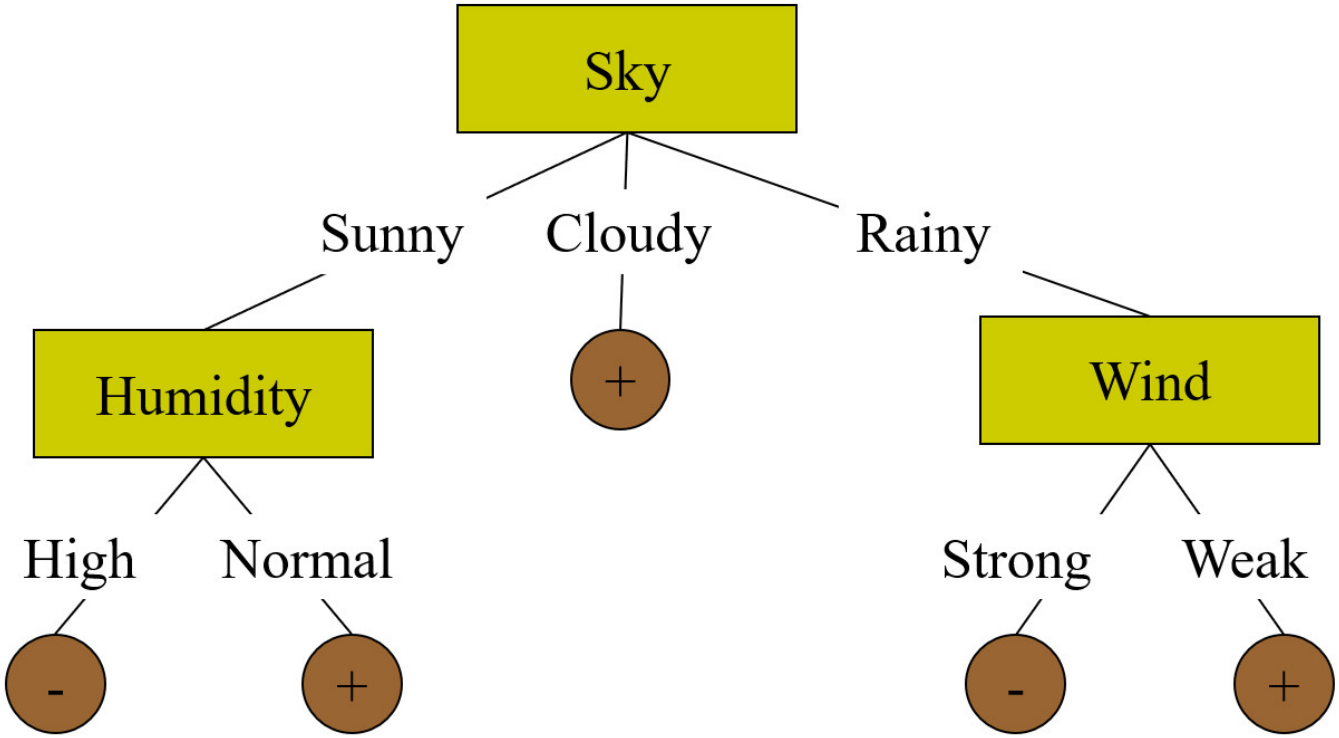
- Cada nodo interno prueba el valor de uno de los atributos de entrada A_i.
- Las ramas de ese nodo están etiquetadas con los posibles valores $v_{i,j}$ del atributo A_i.
- Cada nodo hoja especifica un valor $y_k \in \mathcal{Y}$. Este valor es devuelto por la función.

De esta manera, cada rama completa, es decir, una rama que va desde la raíz hasta un nodo hoja, representa una regla o sentencia de lógica proposicional. El conjunto de todas las ramas completas de un AD representa una función total del espacio de entrada al espacio de salida. El espacio $\mathcal{H}$ es el conjunto de tales funciones.

Ejemplo

Este árbol representa una hipótesis para el problema presentado en la diapositiva 9.

La rama más a la izquierda puede leerse: "si Cielo=soleado y Humedad=alta, entonces JugarTenis es no". Puede verificar que todas las entradas posibles están cubiertas por el árbol. También puede verificar que este árbol clasifica correctamente todos los ítems en el conjunto de datos.



Aprender un Árbol de Decisión

¿Cómo hemos obtenido el árbol en la diapositiva 16?

El árbol se ha aprendido del conjunto de datos en la diapositiva 9 aplicando el algoritmo **ID3**.

Este es un algoritmo muy famoso. Hay muchas extensiones y variantes; sin embargo, presentaremos en detalle solo la versión más simple:

- Solo atributos categóricos
- Sin valores faltantes (es decir, tenemos disponible el valor de cada atributo de cada ítem)

Algoritmo ID3

Entradas: El conjunto de datos *Ejemplos*, la especificación de cuál es el *AtributoObjetivo* y la lista de *Atributos* restantes.

Salida: un árbol de decisión.

El algoritmo es recursivo.
Los **Casos base** son:

- Todos los ejemplos en *Ejemplos* tienen el mismo valor v para *AtributoObjetivo*. Entonces devolver un único nodo etiquetado con v.
- *Atributos* está vacío. Entonces devolver un único nodo etiquetado con el valor más frecuente de *AtributoObjetivo* en *Ejemplos*.

Algoritmo ID3 (cont.)

Paso recursivo:
En caso contrario,

- seleccionar un atributo A de *Atributos*
- crear un árbol con nodo raíz etiquetado con A
- para cada valor v_i de A
  - crear un arco etiquetado con v_i
  - $Ejemplos_i = \{e \in Ejemplos$ tal que $e.A = v_i\}$
  - si $Ejemplos_i$ está vacío, agregar un nodo etiquetado con el valor objetivo más frecuente en *Ejemplos*
  - en caso contrario llamar al algoritmo con $Ejemplos = Ejemplos_i, \quad Atributos = Atributos - A$
  - agregar el resultado al final del arco
- devolver el árbol

La característica distintiva de **ID3** es que usa entropía y GI (ganancia de información) para seleccionar el atributo A.

Entropía

En cada paso, **ID3** intenta obtener clases lo más homogéneas u "ordenadas" posible; y la *entropía* H es una medida del "desorden" o "incertidumbre" de un conjunto.

Así que cuanto menor sea la entropía, mejor es el conjunto.

Supongamos que el atributo objetivo es G y sus posibles valores son $\{v_1, \ldots, v_K\}$.

Denotemos $\mathrm{Ejemplos}(v_i) = \{e \in \mathrm{Ejemplos} \mid e.G = v_i\}$.

La definición formal de entropía es

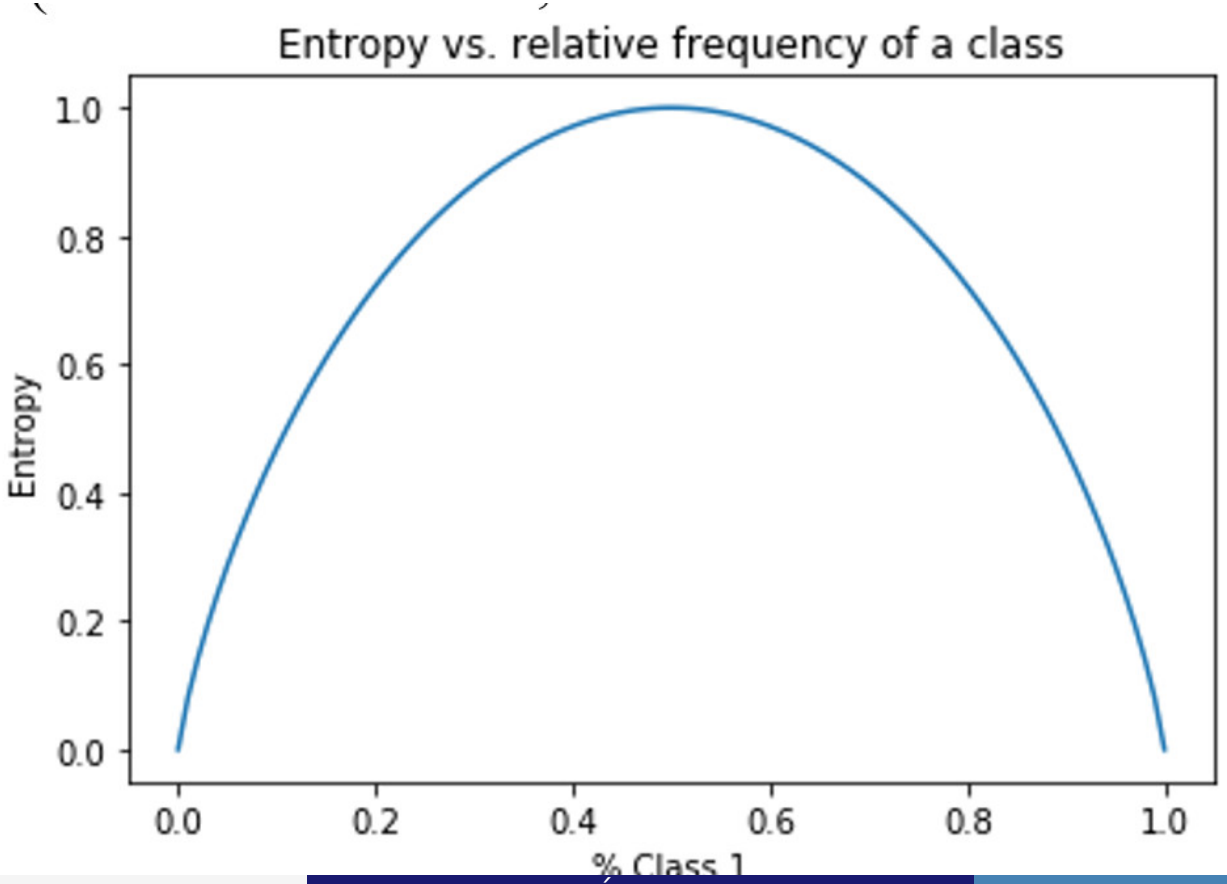
$$H(\mathrm{Ejemplos}) = -\sum_i \frac{|\mathrm{Ejemplos}(v_i)|}{|\mathrm{Ejemplos}|} \log_2 \frac{|\mathrm{Ejemplos}(v_i)|}{|\mathrm{Ejemplos}|}$$

(por favor recuerde que $\lim_{x \leftarrow 0} x \log x = 0$)

Verifique que $H(\mathrm{Ejemplos}) = 0$ si y solo si todos los ejemplos pertenecen a la misma clase, es decir, la clasificación es perfecta.

Entropía (cont.)

Si hay solo dos clases, H es una función de la frecuencia relativa x de, digamos, Clase 1. Si todos los casos pertenecen a la misma clase ($x = 0$ o $x = 1$) entonces $H = 0$ (desorden mínimo, incertidumbre mínima). Por otro lado, si ambas clases son igualmente frecuentes ($x = 0{,}5$) entonces $H = 1$ (desorden máximo, incertidumbre máxima).



Ganancia de Información

La entropía mide la bondad de *un* conjunto. Pero, en cada paso, **ID3** selecciona un atributo A y divide en consecuencia *Ejemplos*, generando *varios* conjuntos.

¿Cómo podemos medir la mejora?

Usamos el concepto de *Ganancia de Información GI*. $\mathrm{GI}(\mathrm{Ejemplos}, A)$ es la diferencia entre la entropía de *Ejemplos* y el promedio ponderado de la entropía de los conjuntos generados. Suponga que los valores de A son $\{w_1, \ldots, w_m\}$. Entonces

$$\mathrm{GI}(\mathrm{Ejemplos}, A) = H(\mathrm{Ejemplos}) - \sum_j \frac{|\mathrm{Ejemplos}(w_j)|}{|\mathrm{Ejemplos}|} H(\mathrm{Ejemplos}(w_j))$$

y **se elige el atributo** A **que produce la mayor** GI.

Ejemplo

Apliquemos **ID3** al conjunto de datos de la diapositiva 9.

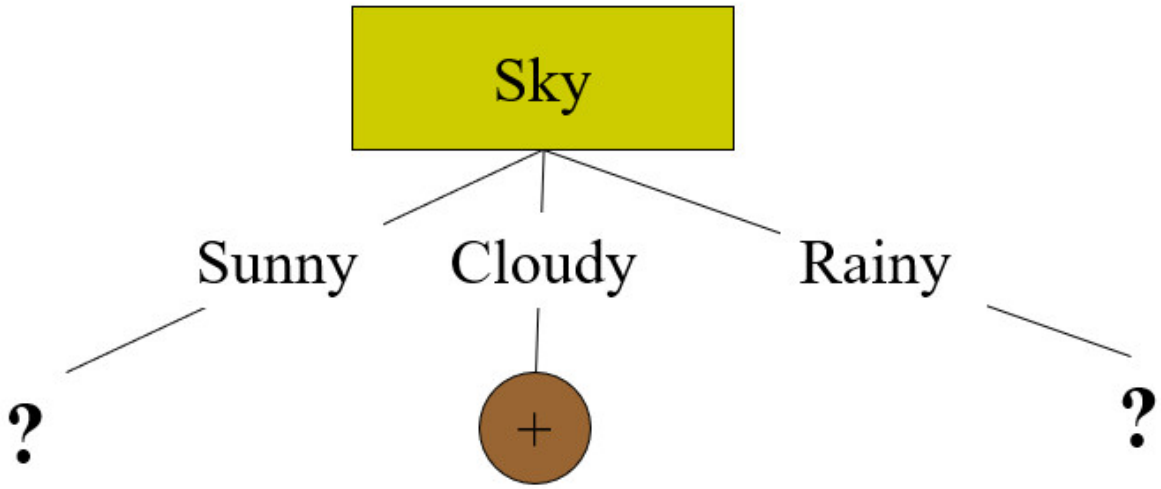
La entropía del conjunto de datos inicial completo es $H(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{14}\}) = 0{,}94$

A	Valores	$Ej(w)$	tam.rel.	H	GI
Humed.	alta	1,2,3,4,8,12,14	0.5	0.985	
	normal	5,6,7,9,10,11,13	0.5	0.592	0.151
Viento	débil	1,3,4,5,8,9,10,13	0.57	0.781	
	fuerte	2,6,7,11,12,14	0.43	1.000	0.048
Cielo	soleado	1,2,8,9,11	0.36	0.971	
	nublado	3,7,12,13	0.28	0.000	
	lluvioso	4,5,6,10,14	0.36	0.971	**0.241**
Temp.	alta	1,2,3,13	0.28	1.000	
	templada	4,8,10,11,12,14	0.43	0.913	
	baja	5,6,7,9	0.29	0.811	0.030

El primer atributo seleccionado es *Cielo*. El nodo raíz está etiquetado con "Cielo", se agregan tres ramas etiquetadas "Soleado", "Nublado" y "Lluvioso", y el conjunto de ejemplos se divide en consecuencia.

Ejemplo (cont.)

El árbol es por el momento



El valor del objetivo para todos los ejemplos en la rama central es "+". Así que se agrega un nodo hoja etiquetado "+".

El algoritmo debe continuar calculando las ramas más a la izquierda y más a la derecha.

Ejemplo (cont.)

Para la rama más a la izquierda *Cielo=Soleado* el conjunto de ejemplos es $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_8, \boldsymbol{x}_9, \boldsymbol{x}_{11}\}$ cuya entropía es $H = 0{,}971$.

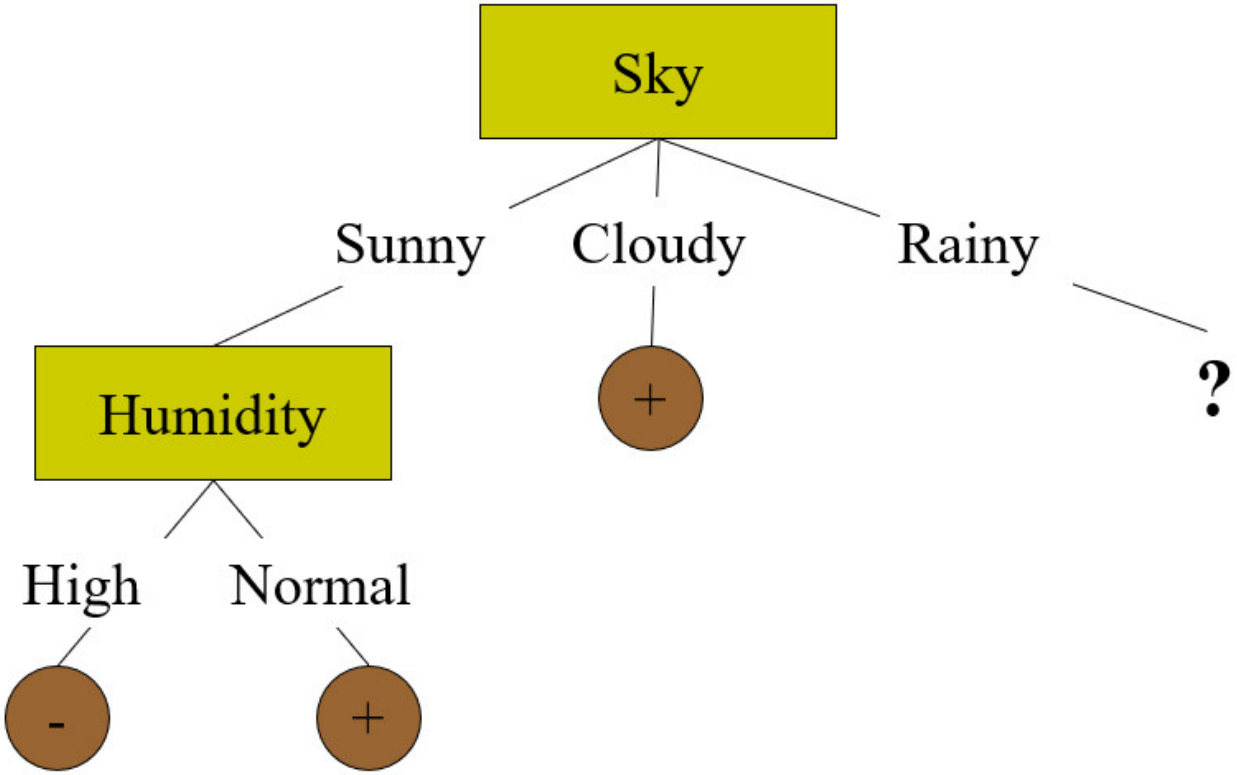
Considerando este conjunto, y los atributos *Humedad, Viento* y *Temperatura*, cálculos análogos a los mostrados en la diapositiva 23 producen

A	*GI*
Humedad	**0.971**
Viento	0.019
Temperatura	0.570

Consecuentemente se agrega un nodo etiquetado con "Humedad", y a él dos ramas etiquetadas "Alta" y "Normal"

Ejemplo (cont.)

El árbol es por el momento



Para ambas ramas de la izquierda, los ejemplos correspondientes son homogéneos. Así que se agregan nodos hoja con las etiquetas adecuadas.

El algoritmo debe continuar calculando la rama más a la derecha.

Ejemplo (cont.)

Para la rama más a la derecha *Cielo=Lluvioso* el conjunto de ejemplos es $\{\boldsymbol{x}_4, \boldsymbol{x}_5, \boldsymbol{x}_6, \boldsymbol{x}_{10}, \boldsymbol{x}_{14}\}$ cuya entropía es $H = 0{,}971$.

Considerando este conjunto, y los atributos *Humedad, Viento* y *Temperatura*, cálculos análogos a los mostrados en la diapositiva 23 producen

A	*GI*
Humedad	0.820
Viento	**0.971**
Temperatura	0.820

Consecuentemente se agrega un nodo etiquetado con "Viento", y a él dos ramas etiquetadas "Fuerte" y "Débil".

Para ambas ramas los ejemplos correspondientes son homogéneos. Así que se agregan nodos hoja con las etiquetas adecuadas.

No quedan más ramas pendientes. El algoritmo devuelve el árbol mostrado en la diapositiva 16

ID3 con profundidad limitada

Opcionalmente, es posible establecer la profundidad máxima D admisible para cualquier rama del AD.

En este caso,

- El algoritmo tiene otro argumento de entrada *maxd*
- El algoritmo se llama con $maxd = D$
- Si *maxd*=0, el algoritmo devuelve un único nodo etiquetado con el valor más frecuente de *AtributoObjetivo* en el argumento presente *Ejemplos*.
- En cada llamada recursiva, el argumento se decrementa en 1.

Características de ID3

Cosas buenas:

- Es simple y fácil de entender.
- Su salida es fácil de entender: cada rama es la conjunción de algunas comprobaciones elementales.
- En la práctica, su rendimiento suele ser bueno.

Por estas razones, es muy popular en libros de texto.

Por otro lado:

- No maneja atributos continuos, ni valores faltantes. Sin embargo, la versión realmente usada en la práctica —llamada **C4.5**— sí puede.
- No hay garantía de que el árbol sea "óptimo" en ningún sentido (tamaño, error). De hecho, la teoría muestra que el problema de encontrar un AD mínimo es NP-completo.

Rendimiento, Entrenamiento y Aprendizaje

Precisión y error de clasificación

¿Qué tan bueno es un clasificador? Para medir su rendimiento, lo aplicamos a un conjunto dado de ejemplos y para cada ejemplo anotamos la clase calculada (*predicha*) por el clasificador y la comparamos con la clase real.

La primera medida obvia de rendimiento es la **precisión** *prec*: número de ejemplos correctamente predichos dividido por el número total de ejemplos.

Es evidente que $\mathrm{prec} \in [0, 1]$ y que cuanto mayor sea la precisión, mejor es el rendimiento.

El complemento de la precisión es la **tasa de error** $\mathrm{err} = 1 - \mathrm{prec}$.

Es evidente que $\mathrm{err} \in [0, 1]$ y que cuanto menor sea la tasa de error, mejor es el rendimiento.

(Consideraremos esta cuestión más generalmente en las próximas lecciones).

Matriz de confusión

A partir de esa información, también podemos compilar la *matriz de confusión* para el clasificador. Suponga que hay M clases $C_1, \ldots, C_M$.

La matriz tiene tamaño $M \times M$. Su elemento (i, j) es el número n_{ij} de ejemplos de clase j que se predicen que pertenecen a clase i.

	Actual class			
Predicted class		1	…	M
	1	n_{11}	…	n_{1M}
	…	…		…
	M	n_{M1}	…	n_{MM}

La precisión es entonces

$$\text{prec} = \frac{\sum_1^M n_{ii}}{\sum_1^M \sum_1^M n_{ij}}$$

Matriz de confusión para clasificadores binarios

Suponga ahora que hay solo dos clases. La Clase 1 se llamará *clase positiva*, y la Clase 2 se llamará *clase negativa*. La matriz de confusión tiene dimensiones 2×2 y sus elementos se llaman

- Verdaderos positivos $VP = n_{11}$
- Verdaderos negativos $VN = n_{22}$
- Falsos positivos $FP = n_{12}$, también llamados errores Tipo I
- Falsos negativos $FN = n_{21}$, también llamados errores Tipo II

	Actual class		
Predicted class		1	2
	1	TP	FP
	2	FN	TN

Entrenamiento y aprendizaje

Como dijimos en la diapositiva 12, generamos una hipótesis h a partir de un conjunto de datos dado E donde tanto las entradas $\boldsymbol{x}$ como las salidas y son conocidas. Esto se llama **entrenamiento** o *ajuste*.

Eso es lo que hemos hecho con los Árboles de Decisión.

Luego calculamos qué tan buena es la hipótesis (en nuestro caso, el AD), es decir, qué tan bien se ajusta al conjunto de datos. Si, por ejemplo, consideramos la tasa de error, obtenemos lo que se llama *error de entrenamiento*.

También lo hemos hecho.

Pero el *aprendizaje* no es solo entrenamiento; ya que queremos usar h para predecir valores de y fuera de E (donde solo conoceremos $\boldsymbol{x}$) debemos evaluar qué tan buena es h al *generalizar*, es decir, cuando h se aplica fuera del conjunto de datos de entrenamiento E para predecir el valor de y.

Sobreajuste

Además, para comparar diferentes clasificadores, el error de entrenamiento NO es una medida confiable.

Quizás piense que cuanto mejor sea el error de entrenamiento, mejor será el rendimiento en todo el espacio de entrada. ¡¡¡NO NECESARIAMENTE!!!

Muchas veces el clasificador se ajusta a los datos de entrenamiento "demasiado bien"; de hecho, tan bien que tiene un rendimiento pobre cuando se aplica a datos no del conjunto de datos de entrenamiento.

Eso se llama **sobreajuste**.

Volveremos a este tema en unas pocas lecciones.

Validación y Prueba

Entonces, ¿qué deberíamos hacer?

La idea es medir el rendimiento de la hipótesis cuando se aplica a pares $(\boldsymbol{x}, y)$ no usados para entrenamiento. Por ejemplo, si consideramos la tasa de error, obtenemos lo que se llama *error de prueba* que es diferente del error de entrenamiento, y usualmente mayor.

La forma más directa de hacer esto es la siguiente:

1. Dividir el conjunto de datos dado en dos piezas, *conjunto de entrenamiento* y *conjunto de prueba*.
2. Usar el conjunto de entrenamiento para obtener un clasificador (en nuestro caso, un AD) (y medir el rendimiento en él, si lo desea).
3. Medir el rendimiento en el conjunto de prueba.
4. Este último es una estimación más confiable del rendimiento en el espacio.

Hay formas más sofisticadas; volveremos a este tema en unas pocas lecciones.

Conclusión

Para resumir

- Al aplicar un algoritmo a un conjunto de datos, se puede generar automáticamente una hipótesis (clasificador, regresor).
- Los Árboles de Decisión (AD) son muy usados como clasificadores.
- El algoritmo **ID3** se usa para generar AD.
- Hay diferentes medidas de rendimiento para clasificadores.
- La prueba es tan importante como el entrenamiento.